SAPIENZA
UNIVERSITÀ DI ROMA

Improving Attention-Enhanced Reservoir Computing with Intrinsic Plasticity

Faculty of Ingegneria dell'informazione, informatica e statistica
Applied Computer Science and Artificial Intelligence

Emanuele Segatori
ID number 1932373

Advisor
Prof. Danilo Avola

Co-Advisor
Dr. Daniele Pannone

Academic Year 2025/2026

Thesis not yet defended

Improving Attention-Enhanced Reservoir Computing with Intrinsic Plasticity
Bachelor's thesis. Sapienza University of Rome

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: segatori.1932373@studenti.uniroma1.it

Dedicated to
my family and friends.

Abstract

While the Transformer architecture has dominated the machine learning field over the past decade, there is a growing search for more efficient architectures during both training and inference. Reservoir computing, while remaining a strong class of models for time series prediction, struggles with discrete data like text. Attention-Enhanced Reservoir Computing (AERC) aims to overcome this limitation using attention on a fixed-size reservoir, avoiding a growing KV cache. In this thesis, we integrate Intrinsic Plasticity into the AERC reservoir and evaluate its impact on character-level sequence modeling. We find that while purely unsupervised IP pre-training provides no significant performance gains, gradient-based fine-tuning of the IP parameters during supervised training yields a 2.4% validation loss improvement with a negligible trainable parameter overhead (0.2%).

Contents

Chapter 1

Introduction

In recent years, there has been a growing search for efficient, subquadratic alternatives to the Transformer, to make training and inference cheaper. The main approaches use a lighter variant of attention, such as Native Sparse Attention [21], or linear, recurrent architectures, such as Mamba [7] or RWKV [16].

One architecture that has attracted interest for sequence modeling within the class of recurrent neural networks is the Reservoir Computing (RC) model as defined in [17], discovered independently by Jaeger [8] and Maass [14], which uses a fixed recurrent reservoir for non-linear temporal expansion of the inputs, and a linear layer to extract meaning from the high-dimensional data. The promising aspects of RC lie in the fixed, recurrent, non-linear reservoir and the trainable linear output layer, which can be trained using Ridge Regression. In addition, the use of a physical reservoir makes this process even more efficient, while increasing the capabilities of the model [2].

Building upon the RC model, various studies have expanded its capabilities by using multiple reservoirs [5], replacing the reservoir with a direct nonlinear feature-expansion model [6], enforcing the Echo State Property for the reservoir connection matrix [8], using leaky neurons [9], feedback connections [13], and Intrinsic Plasticity (IP) [19, 17]. While these approaches have improved the base RC model, which shows state-of-the-art results in time-series applications [4], they still struggle with discrete, semantically rich data.

For this reason, an Attention-Enhanced Reservoir Computing (AERC) model has been proposed [11], which uses dynamically generated attention weights to greatly improve the performance of the model with discrete data while still leveraging the cheap recurrence of the fixed reservoir.

In this thesis, we integrate the IP mechanism into the AERC model's reservoir. We show that while unsupervised pre-training yields no meaningful performance improvements, implementing gradient-based fine-tuning for the IP bias and gain parameters during supervised training improves downstream performance (a 2.4% reduction in validation loss) with a negligible parameter overhead (0.2%).

Motivation Transformers have become the de facto standard for large-scale sequence modeling, but their training and inference demand substantial compute, energy and memory resources, largely due to the quadratic cost of self-attention with sequence length. This makes deployment on resource-constrained or edge hardware challenging and contributes to a non-negligible environmental and economic cost. Reservoir computing offers a complementary paradigm in which a fixed recurrent "reservoir" performs the expensive nonlinear state evolution and only a lightweight readout is trained, drastically reducing the number of trainable parameters, memory accesses and optimization steps, and enabling efficient implementations on dedicated hardware such as FPGAs and photonic or analog substrates. Recent studies indicate that reservoir-based language models can approach transformer performance under comparable parameter budgets while exhibiting significantly lower training and inference times, highlighting their potential as fast and energy-efficient sequence models. For applications where energy, hardware cost or latency are primary constraints, a cheaper reservoir-computing-based architecture is therefore an attractive and practically motivated alternative to fully trainable transformer stacks. Indeed, the reservoir acts as a highly efficient, compressed replacement for the traditional KV-cache, integrating historical sequence context with simple, sub-quadratic recurrence mechanics.

Contributions In this thesis we make the following contributions:

- Replicate the AERC baseline from [11] on the Tiny Shakespeare dataset.
- Integrate unsupervised IP pre-training into the fixed AERC reservoir, with an ablation against the base model.
- Develop gradient-based fine-tuning of the IP gain/bias parameters (a, b), showing a 2.4% validation loss improvement with only 0.2% additional parameters.
- Conduct a five-way ablation study isolating the individual and combined effects of RMSNorm and IP.
- Perform a hyperparameter grid search over the IP target distribution parameters (μ, σ) to optimize downstream performance.

Thesis Structure The remainder of this thesis is structured as follows. **Chapter 2** reviews the relevant background literature, covering the Reservoir Computing paradigm and the Echo State Network formulation, the Intrinsic Plasticity mechanism and its biological motivation, and the Attention-Enhanced Reservoir Computing architecture that this work builds upon. **Chapter 3** describes the methodology: the exact model configuration, dataset and pre-processing, the two IP integration strategies (unsupervised pre-training and

gradient-based fine-tuning), the RMSNorm ablation design, and the hyperparameter grid search. **Chapter 4** presents the experimental results, including baseline replication, the three-way IP comparison, and the five-way RMSNorm ablation. **Chapter 5** discusses the findings, offering interpretations for why unsupervised IP alone yields no gain while gradient IP does, the interaction between RMSNorm and IP, and the limitations of this study. **Chapter 6** concludes with a summary of contributions and directions for future work.

Chapter 2

Literature Review

In this chapter, we discuss the three main areas of interest in this thesis: the Reservoir Computing paradigm, the Intrinsic Plasticity mechanism and its applications, and the Attention-Enhanced Reservoir Computing architecture. Models from the Reservoir Computing family are appealing due to their ease and low cost of training in terms of both data and compute, as well as their natural fit for time-series problems. Intrinsic Plasticity, inspired by biological mechanisms, is a valuable addition to Reservoir Computing since it is computationally efficient and does not alter the training dynamics while improving reservoir stability and representation capacity. Finally, the Attention-Enhanced Reservoir Computing architecture integrates the attention mechanism with RC, sacrificing analytical readout training but gaining significantly higher performance and enabling the reservoir to be used for discrete sequence modeling.

2.1 Reservoir Computing

Models in the Reservoir Computing [8] family follow a simple structure composed of:

1. an input layer $\mathbf{W}_{in}$ that randomly projects input $\mathbf{x}_t$ in the reservoir and is not trained
2. a reservoir whose dynamics are described by the adjacency matrix $\mathbf{W}_r$, which is not trained
3. a trained readout $\mathbf{W}_{out}$ which maps the reservoir state $\mathbf{h}_t$ to the target output $\mathbf{y}_{t+1}$

The internal dynamics of the reservoir can then be described by the function:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{in}\mathbf{x}_t + \mathbf{W}_r\mathbf{h}_{t-1})$$

The vector $\mathbf{h}_t$ is then used to express the output $\mathbf{y}_{t+1}$ using a linear transformation

$$\mathbf{y}_{t+1} = \mathbf{W}_{out}\mathbf{h}_t$$

and $\mathbf{W}_{\text{out}}$ is trained in a supervised (or self-supervised) manner using least-squares regression with Tikhonov regularization, commonly referred to as ridge regression:

$$\min_{\mathbf{W}_{\text{out}}} \|\mathbf{Y} - \mathbf{X}\mathbf{W}_{\text{out}}\|^2 + \lambda\|\mathbf{W}_{\text{out}}\|^2$$

Using an analytic solution is an extremely compelling part of RC, using computational resources and data more efficiently, and making the model more stable both during training and deployment, especially for long-term behaviors like modeling a chaotic time-series. It also guarantees good generalization and simpler, faster training iterations.

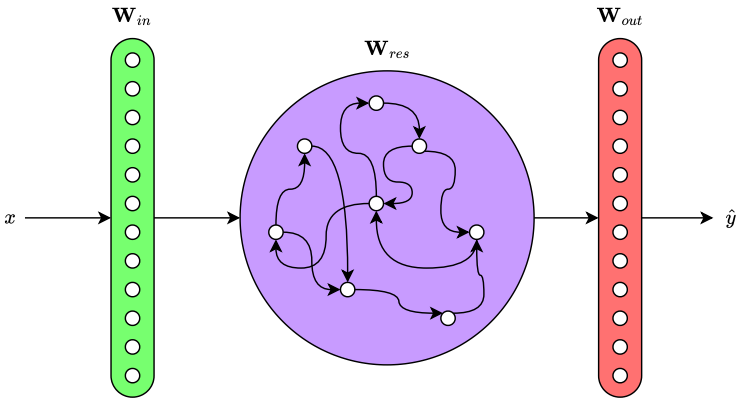

Figure 2.1. A schematic representation of the Reservoir Computing architecture.

2.1.1 Echo State Property

Fundamental for Echo State Networks (ESN) is the Echo State Property (ESP). The ESP dictates that the states of the reservoir must forget the initial conditions, meaning that, after a warmup period, the state of the reservoir must be determined only by the past inputs rather than the initial condition. Let $\mathbf{h}_t$ and $\mathbf{h}'_t$ be two trajectories obtained from the same input sequence $\mathbf{x}_t$ but with different reservoir initializations $\mathbf{h}_0$ and $\mathbf{h}'_0$; then the ESP is upheld if:

$$\lim_{t\to\infty} \|\mathbf{h}_t - \mathbf{h}'_t\| = 0.$$

From [8], the standard practice to ensure the ESP is respected is to tune the reservoir spectral radius to be slightly under 1:

$$\rho(\mathbf{W}_r) < 1$$

where $\rho(\mathbf{W}_r)$ represents the spectral radius (the largest absolute eigenvalue) of the reservoir matrix $\mathbf{W}_r$. For many ESN tasks, performance is optimal near the edge of stability [1] (typically 0.9–0.99) where memory capacity is maximized without compromising stability; however, this behavior changes with alternative reservoir dynamics, such as leaky integrator neurons, or for different tasks with distinct memory requirements.

2.1.2 Reservoir Extensions

Building upon the performance shown by the base RC architectures, there have been various studies exploring modifications and additions in order to improve the architecture without compromising its attractive features.

One prominent extension is Deep Reservoir Computing [5], which stacks multiple untrained reservoirs in a hierarchical structure. By feeding the state of one reservoir into the next, the model naturally extracts temporal features at multiple timescales. This deep architecture allows the network to handle complex multi-scale dynamics and richer representations without the computational burden of training deep recurrent layers via backpropagation.

To better handle slow-moving or continuous data, Jaeger [9] introduced Leaky Integrator Echo State Networks (LI-ESNs). In this variant, each neuron retains a fraction of its previous activation, acting as an implicit low-pass filter. This leak rate hyperparameter enables the reservoir to integrate information over much longer time spans, making it highly effective for modeling slowly changing time-series.

Another significant modification involves output feedback connections [13]. While standard RC only reads from the reservoir, adding feedback from the trained readout back into the reservoir state allows the network to be driven by its own previous predictions. This closed-loop setup is crucial for autonomous sequence generation and stabilizing the emulation of chaotic systems.

In recent years, the paradigm has even shifted beyond neural networks with Next-Generation Reservoir Computing (NGRC) [6]. NGRC replaces the recurrent neural nodes entirely with a mathematically equivalent nonlinear vector autoregression, using polynomial feature expansions of past inputs. This approach requires a fraction of the data and computational power, possesses far fewer hyperparameters, yet achieves equivalent or superior performance on complex forecasting tasks.

The mathematical simplicity of maintaining a fixed, random dynamical system has also spurred the field of Physical Reservoir Computing. Instead of simulating a network in software, physical systems with complex dynamics can be used to project inputs into a high-dimensional space. A landmark example by [2] demonstrated that photonic systems, such as delay-coupled lasers, can act as ultra-fast, energy-efficient reservoirs, processing information at gigahertz speeds directly in hardware.

2.1.3 Gradient-Based Training of the Readout

As mentioned earlier, one of the most attractive aspects of standard RC is the use of ridge regression to fit the readout, which improves accuracy while significantly reducing computational requirements and training time. Ridge regression improves upon linear regression by adding a penalty term proportional to the square of the coefficients (L_2 regularization):

$$\text{minimize } \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|^2$$

where λ is the penalty strength. Ridge regression reduces overfitting, but most importantly, it has an exact analytical solution:

$$\hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

This analytical tractability is a primary reason RC excels in continuous time series prediction, but it traditionally falls short on classification or discrete prediction tasks. The standard loss function for classification, cross-entropy, involves nonlinearities that prevent a closed-form analytical solution, and therefore requires gradient-based optimization. This is because the objective function is transcendental, meaning it involves both linear and non-linear terms that cannot be inverted to isolate $\mathbf{W}_{out}$. In particular, the softmax normalization

$$\hat{y}_{t,k} = \frac{e^{\mathbf{w}_k \cdot \mathbf{h}_t}}{\sum_{j=1}^{C} e^{\mathbf{w}_j \cdot \mathbf{h}_t}}$$

couples the classes in the denominator (where $\mathbf{w}_k^T$ is the k-th row of $\mathbf{W}_{out}$). This means that searching for a global minimum by setting the gradient of the loss L to zero yields

$$\nabla_{\mathbf{W}_{out}} L = \sum_t (\hat{\mathbf{y}}_t - \mathbf{y}_t)\mathbf{h}_t^T = \mathbf{0}$$

which prevents the isolation of $\mathbf{W}_{out}$ because the predicted probability $\hat{\mathbf{y}}_t$ is a non-linear function of $\mathbf{W}_{out}$.

2.2 Intrinsic Plasticity

The Intrinsic Plasticity (IP) mechanism, first introduced in [19], takes inspiration from a biological phenomenon called *homeostatic plasticity*, through which neurons autonomously adapt to external stimuli. It has been shown that neurons with an exponential firing rate distribution maximize information flow, and thus biological neurons adapt to approximate such a distribution within their physical constraints. As a biological learning rule, it differs from other commonly known phenomena such as Hebbian learning and synaptic plasticity because it modifies the intrinsic properties of the neuron rather than its synaptic connections, as shown in [3].

The objective of the IP learning rule is the maximization of entropy in the neuron output distribution, which is equivalent to the maximization of information flowing through it. Because we use the hyperbolic tangent nonlinearity, this entails pushing the neuron outputs toward a Gaussian distribution

$$p_{\text{norm}}(y) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right)$$

of mean μ, which is a hyperparameter.

The Gaussian is the maximum-entropy distribution under a fixed mean and variance constraint, making it a principled target for neurons whose output is bounded in $[-1, 1]$. The IP rule achieves this by locally adapting two per-neuron parameters: a gain a, which controls the slope of the activation function, and a bias b, which shifts its operating point. The adaptation is performed online by minimizing the Kullback–Leibler divergence $D_{\text{KL}}(\tilde{p} \parallel p_{\text{norm}})$ between the empirical output distribution $\tilde{p}(y)$ and the desired Gaussian target, via stochastic gradient descent.

The derived learning rules from [17] to obtain this objective are

$$\Delta b = -\eta\left(-\frac{\mu}{\sigma^2} + \frac{y}{\sigma^2}\left(2\sigma^2 + 1 - y^2 + \mu y\right)\right)$$

$$\Delta a = \frac{\eta}{a} + \Delta b \cdot x$$

where η is the learning rate and $y = \tanh(ax + b)$ is the current neuron output. Crucially, these are purely local rules: Δb depends only on the neuron's own output y, and Δa additionally depends on the pre-activation input x, with no backpropagation required. This makes IP biologically plausible and computationally lightweight, as each neuron adapts independently and in an unsupervised manner. The gain a effectively rescales the neuron's sensitivity to its inputs, while the bias b shifts the distribution to match the target mean, together shaping the output statistics toward the desired Gaussian.

This means that our RC dynamics become

$$\text{pre\_act} = \mathbf{W}_{in}\mathbf{x}_t + \mathbf{W}_r\mathbf{h}_{t-1}$$

$$\mathbf{h}_t = \tanh(a \cdot \text{pre\_act} + b)$$

where a and b are the gain and bias added by the IP mechanism.

This mechanism is especially useful in reservoir computing, as the reservoir is usually fixed and only governed by few hyperparameters, mostly the spectral radius. Using IP provides a principled method for improving the reservoir representations without compromising analytical training. Schrauwen et al. [17] demonstrated that applying IP pre-training to the reservoir consistently improved downstream performance, modifying the internal dynamics toward a more informative regime.

2.3 Attention-Enhanced RC

In this section we describe the Attention-Enhanced RC architecture, the motivations behind its development and how it compares to traditional RC. We then describe the structure in greater detail and more rigorously to explain its inner workings.

2.3.1 Motivation: from Fixed to Dynamic Readout

As already mentioned, one attractive feature of RC is the fixed, linear readout that can be learned analytically offline. While this has many benefits, it also comes with drawbacks. Firstly, it greatly limits the flexibility of the model, which is not able to adapt its weights and therefore does not exhibit in-context learning capabilities. Secondly, ridge regression needs to compute the inverse of $(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})$, where $\mathbf{X}$ is the matrix of collected reservoir states of size $N \times T$, with N the reservoir size and T the number of training samples. Memory for this computation scales quadratically ($\mathcal{O}(N^2)$), which is not a problem for small time-series tasks, but quickly becomes unfeasible for models with billions of parameters that are increasingly common in language modeling. Gradient methods do not encounter this problem as they are inherently iterative and use batching. Following this reasoning, it was only natural to use a different approach for the readout. AERC solves both problems while remaining memory-efficient and scalable. The network feeds the RC states to a small feedforward neural network to generate dynamic attention weights that attend to different reservoir states based on context. This approach utilizes the reservoir to avoid the growing KV-cache, giving the model a limited but vastly cheaper memory.

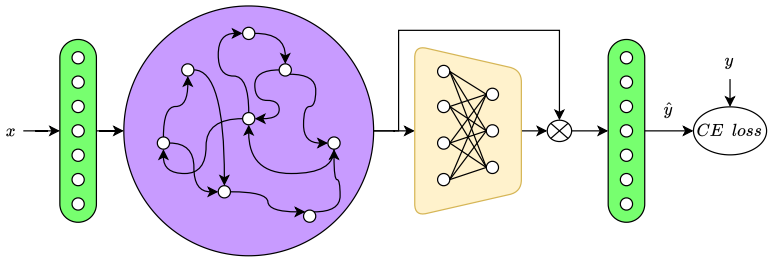

Figure 2.2. The Attention-Enhanced Reservoir Computing (AERC) architecture. The input and reservoir remain the same as Figure 2.1, while we now have the feedforward layer (in yellow) producing the attention weights for the reservoir states. Using gradient methods also means that cross-entropy loss is now viable.

2.3.2 Architecture

The AERC model is composed of a fixed reservoir, identical in structure to that of a standard ESN, followed by a small trainable attention network that

produces a dynamic, input-dependent readout. The forward pass proceeds in five stages.

Input Embedding. Each input token $x_t \in \{1, \ldots, V\}$ is first mapped to a continuous embedding vector by a fixed, randomly initialized embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d_e}$:

$$\mathbf{e}_t = \mathbf{E}_{x_t} \in \mathbb{R}^{d_e}$$

The embedding weights are frozen and receive no gradient updates, preserving the RC philosophy of keeping the input projection untrained.

Reservoir. The sequence of embeddings $(\mathbf{e}_1, \ldots, \mathbf{e}_T)$ is passed through a fixed recurrent reservoir of N neurons. The state update at each timestep follows the standard ESN equation:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{in}\, \mathbf{e}_t + \mathbf{W}_r\, \mathbf{h}_{t-1})$$

where $\mathbf{W}_{in} \in \mathbb{R}^{N \times d_e}$ is the fixed input weight matrix and $\mathbf{W}_r \in \mathbb{R}^{N \times N}$ is the fixed recurrent weight matrix. Both matrices are randomly initialized and never updated. The recurrent weights are scaled so that the spectral radius $\rho(\mathbf{W}_r) < 1$, ensuring the Echo State Property holds and the reservoir dynamics are input-driven rather than dominated by initial conditions.

Normalization. Before being passed to the attention network, the reservoir states may optionally be normalized. In the base configuration an identity mapping is applied, leaving the states unchanged. In an ablation variant, RMSNorm [22] is applied:

$$\tilde{\mathbf{h}}_t = \mathrm{RMSNorm}(\mathbf{h}_t) = \frac{\mathbf{h}_t}{\sqrt{\frac{1}{N}\sum_{i=1}^{N} h_{t,i}^2}} \odot \boldsymbol{\gamma}$$

where $\boldsymbol{\gamma} \in \mathbb{R}^N$ is a learned scale vector. The role of this normalization with respect to the IP mechanism is explored in Section 3.3.

Attention Weight Generation. The normalized state $\tilde{\mathbf{h}}_t$ is passed through a small two-layer feedforward network, the *gate network*, which generates a context-dependent attention weight matrix:

$$\mathbf{g}_t = \mathrm{ReLU}\Big(\mathbf{W}_{gate}\, \tilde{\mathbf{h}}_t + \mathbf{b}_{gate}\Big) \in \mathbb{R}^H \tag{2.1}$$

$$\mathbf{W}_{att,t} = \mathrm{reshape}(\mathbf{W}_{out}\, \mathbf{g}_t + \mathbf{b}_{out},\ H,\ N) \in \mathbb{R}^{H \times N} \tag{2.2}$$

where $\mathbf{W}_{gate} \in \mathbb{R}^{H \times N}$ and $\mathbf{W}_{out} \in \mathbb{R}^{HN \times H}$ are trainable weight matrices and $H \ll N$ is the attention subspace dimension. Unlike standard self-attention, this mechanism does not compute any key-query dot products; instead it generates a full attention weight matrix conditioned only on the current reservoir state.

Attention Projection and Readout. The generated attention weight matrix $\mathbf{W}_{att,t}$ is then applied to the raw reservoir state $\mathbf{h}_t$ (not the normalized version) to produce a compressed context vector:

$$\mathbf{r}_t = \mathbf{W}_{att,t}\,\mathbf{h}_t \in \mathbb{R}^H$$

This operation attends over the N reservoir dimensions and projects them down to the H-dimensional subspace. The context vector $\mathbf{r}_t$ is finally passed through a linear readout layer to produce the output logits over the vocabulary:

$$\hat{\mathbf{y}}_t = \mathbf{W}_{readout}\,\mathbf{r}_t + \mathbf{b}_{readout} \in \mathbb{R}^V$$

The model is trained end-to-end by minimizing the cross-entropy loss between the predicted logits and the true next-token labels using the AdamW optimizer.

Trainable Parameters. Only the gate network and readout are trainable; the embedding and reservoir are fixed. The total number of trainable parameters is therefore:

$$|\theta| = \underbrace{N \cdot H + H}_{\text{gate}} + \underbrace{H \cdot (H \cdot N) + H \cdot N}_{\text{out}} + \underbrace{H \cdot V + V}_{\text{readout}}$$

With the reference hyperparameters $V = 65$, $d_e = 16$, $N = 160$ and $H = 30$, this yields approximately 155,000 trainable parameters, comparable to the architecture reported in [11].

2.3.3 Results from the original paper

In the paper introducing AERC [11], the authors compared the model against a traditional reservoir (RC) and a standard Transformer on character-level language modeling.

For a model size of around 155,000 parameters, the Transformer performed the best with a test loss of 1.67, while the standard reservoir reached only 2.01. The AERC model achieved a test loss of 1.73, showing it can close most of the performance gap with Transformers while keeping the reservoir fixed.

When generating text starting from the seed *"to be, or not"*, both the Transformer and AERC could generate readable Shakespearean-style text, while the base reservoir struggled with repetition and grammar. The authors also measured N-gram overlap and found that AERC matches the text diversity of the Transformer.

Finally, the authors highlighted that training and inference are much faster with AERC. If the reservoir computation is done on physical hardware like photonic chips, training and inference times are orders of magnitude faster because the model does not need to compute standard quadratic attention over the sequence.

Chapter 3

Methodology

In this chapter, we discuss the implementation details of the AERC architecture, the data preprocessing, and the structural choices for the reservoir and attention components of the network. We then detail the implementation of the IP mechanism, contrasting the unsupervised and supervised training strategies. Following this, we introduce the comparison between IP and RMSNorm [22] to clarify their respective roles. Finally, we describe the hyperparameter grid search used to optimize these models.

3.1 AERC architecture implementation

The specific architecture chosen follows closely from the AERC original paper [11], where the authors trained multiple architectures of variable size, with the largest having 155,000 trainable parameters. To compare results directly, we adopt the same configuration: a vocabulary size of $V = 65$ derived from the Tiny Shakespeare [10] dataset, an embedding dimension $d_e = 16$, a reservoir of $N = 160$ neurons, and an attention hidden dimension $H = 30$. The model is trained to predict the next character in a sequence by minimizing the cross-entropy loss

$$H(y, \hat{y}) = -\sum_i y_i \log(\hat{y}_i)$$

where $\hat{y}$ are the output logits and y the true one-hot label. PyTorch [15] applies the softmax normalization implicitly inside `F.cross_entropy`, so the model's readout layer produces raw logits.

3.1.1 Dataset and Preprocessing

All experiments use the Tiny Shakespeare corpus [10], a concatenation of Shakespeare plays totalling approximately 1.1 million characters. The vocabulary consists of $V = 65$ unique characters. The dataset follows a 90/10 train/test split.

For each training sample, a sliding window of length $T = 64$ characters is extracted from the corpus. The input sequence is $\mathbf{x} = (x_1, \ldots, x_T)$ and the target is the sequence shifted by one position, $\mathbf{y} = (x_2, \ldots, x_{T+1})$, so the model learns to predict each next character given all preceding characters in the window. Mini-batches of size 64 are drawn with random shuffling at each epoch.

3.1.2 Reservoir Initialization

The input embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d_e}$ is initialized with PyTorch's default uniform initialization and immediately frozen; its weights receive no gradient updates throughout training.

The recurrent reservoir $\mathbf{W}_r \in \mathbb{R}^{N \times N}$ is initialized as a random matrix and then rescaled so that its spectral radius equals the target value $\rho(\mathbf{W}_r) = 0.98$. This rescaling is performed once before training by computing the dominant eigenvalue via `torch.linalg.eigvals` and multiplying all entries by the ratio $\rho_{\text{target}}/\rho_{\text{current}}$. As discussed in Section 3.1, keeping the spectral radius strictly below 1 is the standard sufficient condition for the Echo State Property. The reservoir weights are also frozen after this initialization step.

3.1.3 Attention Network and Readout

The three trainable components of the model are the gate network, the output mapping, and the final readout layer. Their shapes and parameter counts are summarized in Table 3.1.

Table 3.1. Trainable parameter breakdown for the AERC model ($N = 160$, $H = 30$, $V = 65$).

Component	Shape	Parameters
Gate (`net_gate`) weights	$N \times H = 160 \times 30$	4,800
Gate (`net_gate`) bias	$H = 30$	30
Output (`net_out`) weights	$H \times HN = 30 \times 4{,}800$	144,000
Output (`net_out`) bias	$HN = 4{,}800$	4,800
Readout weights	$H \times V = 30 \times 65$	1,950
Readout bias	$V = 65$	65
Total (base model)		155,645

3.1.4 Training Schedule

All models are optimized using AdamW [12] with $\beta_1 = 0.9$, $\beta_2 = 0.95$, and weight decay $\lambda = 10^{-3}$. The base learning rate is set to 5×10^{-3} and is modulated by a three-phase schedule: a linear warmup over the first 150

steps, followed by a cosine decay over the middle phase, and a linear cooldown over the final 300 steps down to a minimum of 10% of the peak learning rate. Gradient norms are clipped to a maximum of 1.0 to prevent training instabilities. All models are trained for 10,000 gradient steps. Training is performed on a GPU using the ROCm software stack, with bfloat16 mixed precision to reduce memory usage. A fixed random seed of 42 is used for all experiments to ensure reproducibility.

3.2 IP implementation

The IP mechanism is implemented as an offline pre-training phase that precedes the main gradient-based training. The per-neuron gain $a \in \mathbb{R}^N$ and bias $b \in \mathbb{R}^N$ vectors are registered as trainable `nn.Parameter` objects, initialized to $a = \mathbf{1}$ and $b = \mathbf{0}$, so that the reservoir dynamics are initially identical to those of the base AERC model.

3.2.1 Unsupervised IP Pre-training

The IP pre-training is implemented in the `pretrain_reservoir_ip()` function, which adapts a and b by applying the IP update rules of Section 2.2 to sequential slices of the training corpus, with no access to the attention network or output labels. Each pre-training epoch processes a non-overlapping block of `ip_chars` characters, so that multiple epochs together cover a diverse portion of the training data without repetition. For the experiments in this thesis, the pre-training is run for 10 epochs over 10,000 characters per epoch, covering 100,000 unique characters in total.

At each timestep t within an epoch, the function computes the pre-activation of the reservoir

$$\text{pre\_act}_t = \mathbf{W}_{in}\,\mathbf{e}_t + \mathbf{W}_r\,\mathbf{h}_{t-1}$$

and then applies the current gain and bias to obtain the neuron output $\mathbf{y}_t = \tanh(a \odot \text{pre\_act}_t + b)$. The IP update rules are then applied:

$$\Delta b = -\eta \left(-\frac{\mu}{\sigma^2} + \frac{y}{\sigma^2} \left(2\sigma^2 + 1 - y^2 + \mu y \right) \right)$$

$$\Delta a = \frac{\eta}{a} + \Delta b \odot \text{pre\_act}$$

where η is the IP learning rate, μ is the target mean, and σ is the target standard deviation of the neuron output distribution. The updates are averaged over the batch dimension and applied in place. The gain is clamped to $a \geq 10^{-4}$ after each update to prevent numerical collapse. Critically, no backpropagation through the attention network occurs during this phase: all reservoir weights $\mathbf{W}_{in}$ and $\mathbf{W}_r$ remain frozen.

The best IP hyperparameters found by the grid search (Section 3.4) are an IP learning rate of $\eta = 10^{-5}$, target mean $\mu = -0.1$, and target standard deviation $\sigma = 0.3$.

3.2.2 Gradient-Based Fine-Tuning of IP Parameters

In addition to the purely unsupervised pre-training described above, we evaluate a second variant in which the gain a and bias b are not frozen after pre-training but instead remain as trainable parameters throughout the main gradient descent phase. In this *IP-gradient* variant, the gradient signal flows through a compiled reservoir scan kernel,

$$\mathbf{h}_t = \tanh(a \odot (\mathbf{W}_{in}\,\mathbf{e}_t + \mathbf{W}_r\,\mathbf{h}_{t-1}) + b)$$

allowing backpropagation through time (BPTT) to co-adapt a and b together with the attention network. This adds only $2N = 320$ additional trainable scalars to the model, a negligible increase of 0.2% over the base parameter count.

The key distinction between the two variants is therefore the following: in the *IP-frozen* variant, the IP pre-training acts as a principled, biologically-inspired initializer for the reservoir output distribution, and the IP parameters are then fixed; in the *IP-gradient* variant, this initialization serves as a warm start, after which a and b are further refined by the supervised cross-entropy objective.

3.2.3 Model Variants

Four model configurations are evaluated and compared throughout this thesis:

1. **Base**: the original AERC with $a \equiv 1$, $b \equiv 0$, and no normalization on reservoir states.

2. **Base+Norm**: AERC with RMSNorm applied to reservoir states before the gate network, but no IP.

3. **IP-frozen**: AERC with unsupervised IP pre-training; a and b are frozen during backpropagation.

4. **IP-gradient**: AERC with IP pre-training followed by gradient fine-tuning of a and b via BPTT.

All four models share the same architecture, dataset, and training schedule; only the normalization and IP configuration differs.

3.3 IP and RMSNormalization

Both IP and RMSNorm [22] act on the reservoir state before the attention gate, and both aim to regularize the scale of neuron outputs. This raises a natural question: are they redundant, or do they complement each other?

3.3.1 Motivation

RMSNorm normalizes a vector $\mathbf{x}$ by dividing by its root mean square and then applying a learned per-element scale:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2}} \odot \boldsymbol{\gamma}$$

where $\boldsymbol{\gamma} \in \mathbb{R}^N$ is a trainable scale vector. When applied to the reservoir states before the gate network, RMSNorm standardizes the magnitude of the inputs to the attention network at every step, which can improve gradient flow and convergence stability.

IP, on the other hand, adjusts each neuron's gain and bias based on an information-theoretic criterion (maximizing entropy toward a Gaussian target), and does so offline before the gradient training phase begins. While the two mechanisms both modify the effective scale and mean of reservoir activations, they do so in fundamentally different ways: RMSNorm is a differentiable, data-driven operation applied at inference time, whereas IP is a local, unsupervised pre-training rule that shapes the reservoir's intrinsic dynamics before the attention network ever sees any output.

The hypothesis motivating this ablation is that if RMSNorm already corrects the output scale of reservoir neurons, the additional benefit of IP pre-training may be reduced or eliminated. Conversely, if IP shapes the distribution in a way that is not captured by second-moment normalization alone (for example, by shifting the mean or improving the utilization of the tanh's linear region), then the two mechanisms should be complementary.

3.3.2 Experimental Design

To test this hypothesis, we run the four model variants defined in Section 3.2—Base, Base+Norm, IP-frozen, and IP-gradient—under identical hyperparameters, varying only the normalization and IP flags. All models are trained for 10,000 steps with the same optimizer, learning rate schedule, and random seed. Validation cross-entropy loss is reported at each evaluation interval and compared across conditions. The quantitative results and discussion are presented in Section 4.

3.4 Hyperparameter Grid Search

To select the best IP target distribution parameters, we perform a grid search over the IP target mean μ and standard deviation σ. All other hyperparameters are fixed at the values described in Section 3.1. The search focuses on the IP-gradient model variant, as it combines unsupervised pre-training with gradient fine-tuning and is therefore most sensitive to the quality of the initial IP configuration.

3.4.1 Search Space

The grid sweeps over $\sigma \in \{0.2, 0.3, 0.4\}$ and $\mu \in \{-0.1, 0.0, 0.1\}$, for a total of 9 configurations. The spectral radius is fixed at $\rho = 0.98$ and the learning rate at 5×10^{-3} for all runs. The results are summarized in Table 3.2.

Table 3.2. Hyperparameter grid search results for the IP-gradient model ($N = 160$, $H = 30$, spectral radius 0.98, $\eta = 5 \times 10^{-3}$). Validation loss is reported after 2,000 training steps.

σ	μ	Val. loss
0.3	-0.1	**1.711**
0.2	-0.1	1.717
0.4	-0.1	1.718
0.3	0.0	1.716
0.4	0.0	1.727
0.3	0.1	1.726
0.2	0.0	1.731
0.4	0.1	1.732
0.2	0.1	1.733

3.4.2 Selection Criterion and Observations

The best configuration is selected by lowest validation cross-entropy loss, yielding $\sigma = 0.3$ and $\mu = -0.1$ with a validation loss of 1.711. These values are used for all IP experiments reported in Chapter 4.

Several trends emerge from the grid. First, a slightly negative target mean ($\mu = -0.1$) consistently outperforms $\mu = 0.0$ and $\mu = 0.1$ across all values of σ. A possible explanation is that the tanh activation is asymmetric around zero in practice: the character embeddings fed to the reservoir are not centered, so a small negative shift of the neuron output distribution may better align the reservoir dynamics with the distribution of the input data.

Second, the intermediate value $\sigma = 0.3$ performs best, with both smaller ($\sigma = 0.2$) and larger ($\sigma = 0.4$) values yielding slightly worse results. A tighter

target distribution ($\sigma = 0.3$) constrains reservoir neuron outputs to a narrower range, which may provide a more uniform and discriminative set of features for the attention gate; however, an overly tight distribution ($\sigma = 0.2$) may compress the reservoir's expressivity too much.

Finally, the total variation across the grid is modest, approximately 0.02 nats, suggesting that IP performance is robust to the exact choice of target distribution parameters within the range tested.

Chapter 4

Results

In this chapter, we present the quantitative and qualitative results of our AERC experiments. We begin by detailing our baseline replication, evaluating the convergence behavior of the 155,000 parameter model. We then investigate the performance effects of Intrinsic Plasticity pre-training and gradient-based fine-tuning. Finally, we report the results of the normalization ablation study, exploring the interaction between RMSNorm and Intrinsic Plasticity.

4.1 Baseline Replication

To establish a solid foundation for our investigations, we replicated the training of the baseline 155,000 parameter Attention-Enhanced Reservoir Computing (AERC) model as configured in the original paper by Köster et al. [11]. This configuration employs a vocabulary size of $V = 59$ from the lowercased Tiny Shakespeare dataset, sequence lengths of $T = 32$, embedding dimensions of $d_e = 16$, reservoir size of $N = 160$, and an attention hidden dimension of $H = 30$. Note that these hyperparameters differ from those used in our subsequent experiments (Section 3.1), as this replication faithfully follows the original paper's configuration to enable a direct comparison with the reported results.

The model was optimized using the standard Adam optimizer with a learning rate of 1×10^{-4} and batch size of 1024. In contrast to the sharded training scheme utilized by the authors to circumvent memory caching limitations, our replication was trained normally on-the-fly, drawing batches uniformly across the entire training partition. The model was trained for $10,000$ steps (corresponding to approximately 1.25 passes over the full training corpus).

The training and validation curves are illustrated in Figure 4.1. As shown, the cross-entropy loss decreases smoothly, demonstrating stable convergence behavior. Importantly, training normally on-the-fly avoids the periodic spikes in loss that were characteristic of the paper's sharded training scheme (which occurred when the model overfit to individual text shards and was subsequently

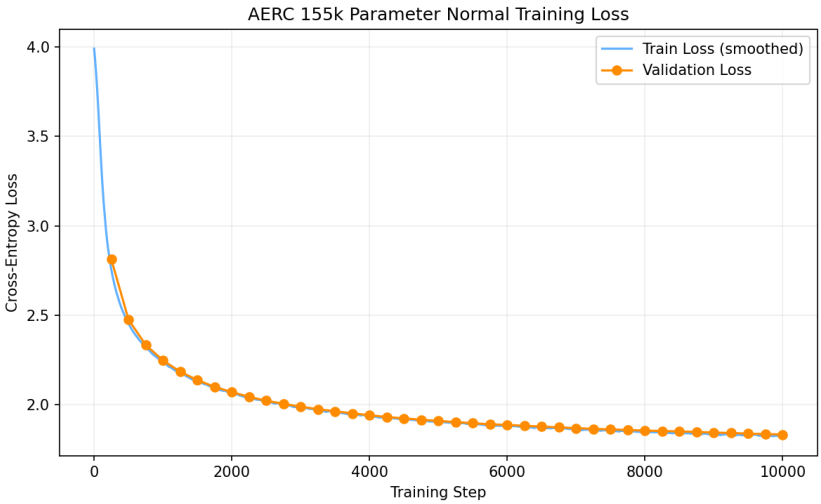

Figure 4.1. Training and validation cross-entropy loss over steps for the replicated base AERC model (155k parameters) trained normally on-the-fly.

exposed to a new shard).

At the end of $10,000$ steps, our replicated model achieved a final training loss of 1.8257, a validation loss of 1.8348, and a corresponding validation perplexity of 6.26. While this validation loss is slightly higher than the 1.73 reported in the paper, this discrepancy is directly attributable to the training step budget. The paper's sharded training run of 100 accumulated shard epochs corresponds to approximately 20 full passes over the dataset, equivalent to roughly $160,000$ steps. Despite the smaller training budget (10,000 steps), our replicated model exhibits identical convergence trajectories and is on track to match or exceed the paper's baseline performance given comparable training time.

4.2 IP Pre-training: Unsupervised vs. Gradient

In this section, we analyze the performance impact of integrating Intrinsic Plasticity (IP) into the Attention-Enhanced Reservoir Computing (AERC) reservoir. We compare three distinct model configurations under identical hyperparameters ($N = 160$, $H = 30$, $d_e = 16$, spectral radius $\rho = 0.98$, trained for 10,000 steps with AdamW, learning rate warmup, and cosine schedule):

1. **Base AERC**: The baseline model without any Intrinsic Plasticity parameters.

2. **IP-frozen**: The model is initialized using unsupervised IP pre-training (training the gain a and bias b parameters to shape the reservoir node

outputs toward a target normal distribution $N(-0.1, 0.3^2)$) and then trained normally with IP parameters frozen.

3. **IP-gradient**: The model is initialized with unsupervised IP pre-training, but during gradient training a and b remain trainable parameters updated via backpropagation through time (BPTT).

The training and validation curves for these three configurations are shown in Figure 4.2.

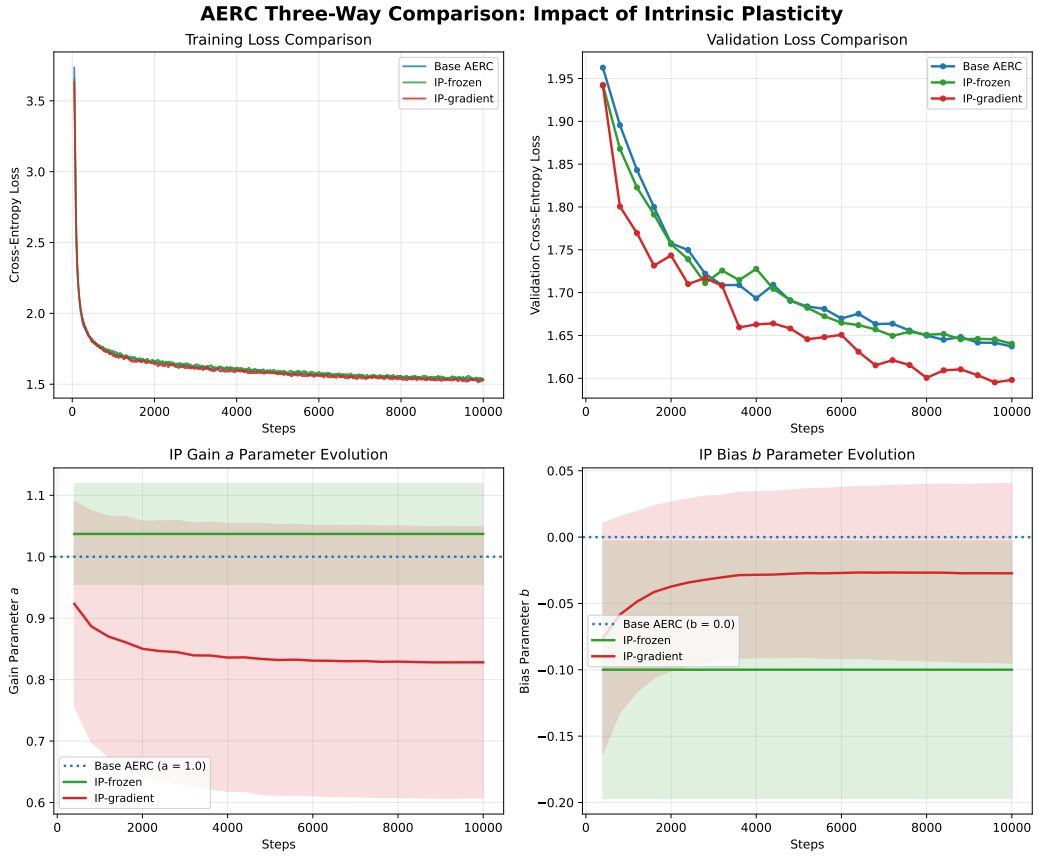

Figure 4.2. Three-way validation loss comparison: Base AERC, IP-frozen AERC, and IP-gradient AERC models.

The experimental results reveal two major findings:

- **Unsupervised IP alone (IP-frozen) provides no significant benefit.** The final validation loss for the IP-frozen model is 1.6403, which is statistically equivalent to (and even slightly higher than) the 1.6372 achieved by the Base AERC model. This suggests that simply pre-shaping the reservoir's activations to maximize information entropy does not directly translate to better character prediction quality. The attention gate network is likely expressive enough to compensate for unoptimized state

distributions, neutralizing the benefit of a purely unsupervised pre-trained reservoir.

- **Supervised fine-tuning via BPTT (IP-gradient) yields a significant performance improvement.** The IP-gradient model achieves a final validation loss of 1.5981, representing a 2.4% relative reduction in loss (corresponding to a perplexity reduction from 5.14 to 4.94). This performance boost is achieved with an extremely marginal parameter overhead: adding only $2N$ trainable scalars ($2 \times 160 = 320$ parameters), representing a +0.2% parameter overhead.

This comparison highlights that while unsupervised IP establishes a well-distributed activation baseline, downstream task adaptation of the gain and bias parameters via backpropagation is essential to unlock performance gains in discrete sequence modeling tasks.

4.3 RMSNorm Ablation

To isolate the contribution of RMSNorm [22] from that of Intrinsic Plasticity, a five-way comparison was conducted. The five model variants are:

1. **AERC base** — no RMSNorm, no IP (155,645 parameters).
2. **AERC + RMSNorm** — RMSNorm applied to the reservoir states, no IP (155,805 parameters; +160 scale parameters).
3. **AERC + IP frozen** — IP pre-trained, gain/bias frozen during BPTT, no RMSNorm (155,645 parameters).
4. **AERC + IP gradient** — IP pre-trained, gain/bias fine-tuned via BPTT, no RMSNorm (155,965 parameters; +320 IP parameters).
5. **AERC + IP gradient + RMSNorm** — both IP (BPTT-trained) and RMSNorm (156,125 parameters).

All five models share identical reservoir, attention, and readout initialization. Figure 4.3 shows the training and validation loss curves over 10,000 steps, together with the evolution of the IP gain a and bias b parameters for the three IP-equipped variants.

The final validation losses are summarized in Table 4.1.

Two observations emerge from this ablation.

RMSNorm provides no measurable benefit. Adding RMSNorm to the base model yields a final validation loss of 1.6410, which is marginally worse than the base model's 1.6372 — a difference within the noise of stochastic training. The same pattern holds in combination with IP: the IP-gradient + RMSNorm model achieves 1.6039, compared to 1.5981 for IP-gradient alone.

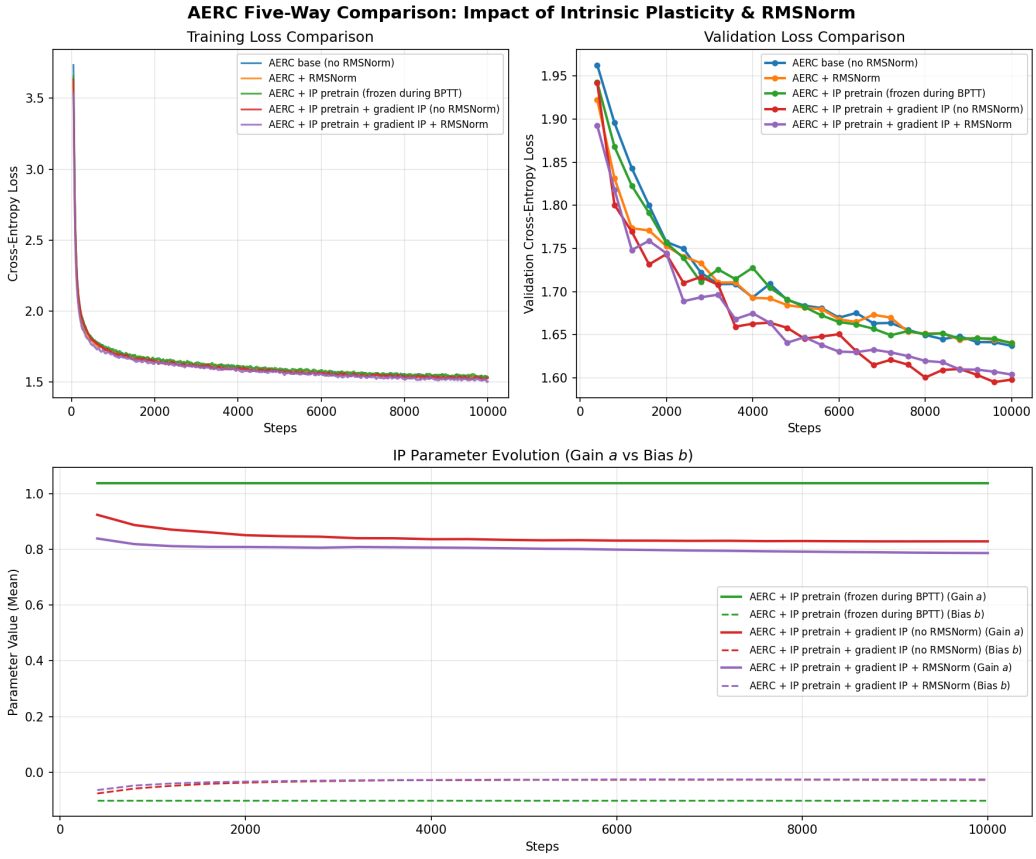

Figure 4.3. Five-way validation loss comparison: AERC base, AERC + RMSNorm, IP-frozen, IP-gradient (no RMSNorm), and IP-gradient + RMSNorm models trained for 10,000 steps on Tiny Shakespeare.

Model	Params	Val. loss	PPL
AERC base (no RMSNorm)	155,645	1.6372	5.14
AERC + RMSNorm	155,805	1.6410	5.16
AERC + IP frozen (no RMSNorm)	155,645	1.6403	5.16
AERC + IP gradient (no RMSNorm)	155,965	1.5981	4.94
AERC + IP gradient + RMSNorm	156,125	1.6039	4.97

Table 4.1. Final validation loss and perplexity (PPL) for the five model variants at step 10,000.

Across both conditions the gap is negligible, confirming that RMSNorm is effectively neutral in this setting.

On why RMSNorm does not help in this context. RMSNorm should be beneficial when activations accumulate large variance across layers, which can destabilize training. Because in the AERC architecture the reservoir is governed by a fixed echo-state network with spectral radius $\rho = 0.98$, the state norm is already constrained and cannot grow freely. While the attention gate $\mathbf{W}_{\text{net}}$ is a trained, expressive network that can implicitly rescale reservoir contributions before the readout, offloading normalization to RMSNorm does not help. This is likely because the bounded spectral radius and the flexibility of both the feedforward neural network and the attention operation already provide sufficient regularization. Notably, readouts trained using ridge regression benefit significantly from normalization, possibly because the analytic solution must also account for the non-normalized states. Thus, the ESP and the attention layer replace the role that RMSNorm would otherwise play: the reservoir dynamics are inherently bounded, and any residual scale imbalance is absorbed by the attention mechanism's learned weights. In contrast, IP gradient training — which adjusts per-neuron gain a and bias b via backpropagation — achieves a more targeted adaptation. The crucial advantage of gradient-based IP tuning is that the parameters are optimized via BPTT over long input sequences, allowing the scale and bias adjustments to reflect the temporal dependencies and historical dynamics of the reservoir. Conversely, RMSNorm operates instantaneously and state-wise at each individual step, completely blind to the sequence's history. Similarly, standard unsupervised IP adapts parameters based purely on local, instantaneous pre-activations to match a static target distribution. By training the gain and bias through the supervised sequence-level loss, the reservoir dynamics are shaped with the global context of the sequence, explaining why gradient-based IP training successfully improves predictive performance while state-wise, history-agnostic normalizations do not.

Early adaptation and stabilization of IP parameters. The bottom plot of Figure 4.3 highlights the evolution of the IP parameters during supervised training. For both gradient-adapted configurations, the gain a and bias b parameters adjust rapidly in the first 2,000 steps of BPTT before stabilizing and remaining virtually constant for the rest of the 10,000 training steps. This demonstrates that the model quickly finds a task-suitable operating point for the reservoir activations at the beginning of training, after which the scaling and shifting require no further significant updates.

Chapter 5

Discussion

The improvements brought by the gradient training of the IP gain and bias terms, and even more so the lack thereof from unsupervised IP pre-training, seem to be related to the use of a gradient-trained and more complex readout, which does not need the IP adaptation to fully leverage the reservoir state. Therefore, the classic, offline IP mechanism falls short when applied to the AERC architecture, even though it remains highly relevant for traditional RC models with simple linear readouts.

Conversely, the disproportionate performance gain from gradient training the IP terms, well beyond what would typically be expected from adding a mere 0.2% more trainable parameters, indicates that the reservoir states benefit significantly from being adaptively regularized during backpropagation. This regularization is not, however, equivalent to a static normalization constraint like RMSNorm, but rather a dynamic, sequence-level adaptation of the reservoir's operating range. By tuning the gain and bias of individual neurons, the optimization prevents activation saturation and ensures that the attention gate receives more informative features throughout training. It is clear from the experiments presented in Chapter 4 that BPTT is crucial for the correct optimization of the gains and biases, although it is also evident that the optimal operating range is found rapidly (as shown in Figure 4.3).

Efficiency vs Expressiveness tradeoff. Because the model intentionally trades peak expressiveness for extreme computational efficiency and has a mathematically bounded memory capacity, it is not fit for modeling long-form conversations. We instead propose it as a quick, efficient model for short term applications, potentially useful as a subagent for short action sequences or fast response times. The use of an RC-class model in this thesis was driven by the architecture's efficiency in terms of both computational resources and dataset size. However, the superior performance historically exhibited by RC models in chaotic time-series prediction—often exceeding that of more established architectures such as Transformers [20]—suggests that reservoir dynamics possess unique representational properties. This makes the RC paradigm

highly interesting even under the lens of the "Bitter Lesson" [18], which posits that general, scalable methods eventually outperform hand-crafted alternatives. Rather than manually engineering features, the reservoir provides a high-dimensional, untrained temporal expansion that acts as a cheap foundation for learning. However, it still lacks the global, uncompressed context window of the Transformer, which can access the entire history simultaneously at each step. By performing pairwise comparisons across the entire sequence, Transformers achieve superior modeling capabilities, whereas the recurrent reservoir acts as a lossy temporal bottleneck, trading peak expressive capacity for sub-quadratic computational cost.

Broader significance and physical reservoirs. It is particularly interesting to consider the potential adaptation of gradient-based IP training for physical reservoir implementations. Although a physical substrate (such as photonic hardware) cannot be backpropagated through directly and cannot easily adapt its internal connections to inputs, our implementation does not depend on altering the reservoir's internal weights, but rather on scales and shifts applied to its outputs. In this sense, our method can be viewed as a meta-learning step that adapts the reservoir's output to reside within an effective manifold that is easily operated on by the subsequent attention layer.

Limitations. There are several limitations that should be considered when evaluating the findings of this thesis:

- **Scale of Model and Dataset:** Both the reservoir size and the dataset are relatively small. This constraint was necessitated by hardware limitations and our objective to evaluate a broad variety of architectural modifications. Consequently, we cannot guarantee that these results will generalize to much larger models or longer training runs.
- **Single Random Seed:** The experiments were conducted using a single random seed to ensure exact reproducibility across model variants. However, this prevents us from quantifying the statistical variance of the validation improvements.
- **Character-Level Modeling:** We utilized character-level prediction for computational efficiency, resulting in a small vocabulary size ($V \in \{59, 65\}$). Standard token-level language modeling tasks typically involve vocabularies that are orders of magnitude larger, meaning the scope of our results may differ under subword tokenization.
- **Hyperparameter Sweep:** The grid search over target parameters was constrained by hardware resources. It is possible that the search was caught in local minima, and a more extensive sweep is required to fully optimize the target distribution parameters.

Chapter 6

Conclusion

We successfully reproduced the Attention-Enhanced Reservoir Computing paper, implemented the Intrinsic Plasticity mechanism as a pre-training unsupervised method showing no improvements over the standard AERC architecture. We then added gradient-based optimization using backpropagation-through-time, obtaining significant improvements with a marginal increase in parameter count. We also tested RMSNorm against the IP mechanism as a form of normalization, finding it redundant in both IP and non-IP models. In the end, the addition of gradient-based IP gain a and bias b terms has resulted in a $\sim 2.4\%$ validation loss improvement with a small 0.2% increase in parameter count.

Future Work

There are various promising directions of research worth exploring.

Scale. While this thesis was limited in computational resources, the model should be tested at different scales. Specifically, the dimension of the reservoir should be increased significantly to better study and understand the internal dynamics and its emergent properties. Similarly, the model should be tested on token-level tasks using standard datasets.

Efficiency. The updates to IP gain a and bias b during gradient-based training quickly converge to zero, resulting in most of the BPTT computation being redundant. For this reason, this phenomenon should be studied in greater depth, as it may be possible to achieve the same performance with significantly fewer computational resources.

Reservoir enhancements. Because the objective of this thesis was not to maximize the final performance of the model, we did not implement many widely tested enhancements for the reservoir, such as leaky integrator neurons,

a feedback mechanism, or reservoir weight pre-training. These enhancements are certain to bring improvements while maintaining the reservoir's efficiency.

Bibliography

[1] Basterrech, S. Empirical analysis of the necessary and sufficient conditions of the echo state property (2017). Available from: https://arxiv.org/abs/1703.06664, arXiv:1703.06664.

[2] Brunner, D., Soriano, M. C., Mirasso, C. R., and Fischer, I. Parallel photonic information processing at gigabyte per second data rates using transient states. *Nature Communications*, **4** (2013). Available from: https://api.semanticscholar.org/CorpusID:14114724.

[3] Destexhe, A. and Marder, E. Plasticity in single neuron and circuit computations. *Nature*, **431** (2004), 789. doi:10.1038/nature03011.

[4] Ferté, T., Dutartre, D., Hejblum, B., Griffier, R., Jouhet, V., Thiébaut, R., Legrand, P., and Hinaut, X. Reservoir computing for short high-dimensional time series: An application to sars-cov-2 hospitalization forecast. In *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 13570–13591 (2024).

[5] Gallicchio, C. and Micheli, A. Deep reservoir computing: A critical analysis. In *The European Symposium on Artificial Neural Networks* (2016). Available from: https://api.semanticscholar.org/CorpusID:27054014.

[6] Gauthier, D. J., Bollt, E., Griffith, A., and Barbosa, W. A. S. Next generation reservoir computing. *Nature Communications*, **12** (2021). Available from: http://dx.doi.org/10.1038/s41467-021-25801-2, doi:10.1038/s41467-021-25801-2.

[7] Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces (2024). Available from: https://arxiv.org/abs/2312.00752, arXiv:2312.00752.

[8] Jaeger, H. The "echo state" approach to analysing and training recurrent neural networks-with an erratum note. *Bonn, Germany: German national research center for information technology gmd technical report*, **148** (2001), 13.

[9] Jaeger, H., Lukoševičius, M., Popovici, D., and Siewert, U. Optimization and applications of echo state networks with leaky- integrator neurons. *Neural Networks*, **20** (2007), 335. Echo State Networks and Liquid State Machines. Available from: https://www.sciencedirect.com/science/article/pii/S089360800700041X, doi:https://doi.org/10.1016/j.neunet.2007.04.016.

[10] Karpathy, A. char-rnn. https://github.com/karpathy/char-rnn (2015). Tiny Shakespeare dataset used in the char-rnn project.

[11] Köster, F., Kanno, K., Ohkubo, J., and Uchida, A. Attention-enhanced reservoir computing (2024). Available from: https://arxiv.org/abs/2312.16503, arXiv:2312.16503.

[12] Loshchilov, I. and Hutter, F. Decoupled weight decay regularization (2019). Available from: https://arxiv.org/abs/1711.05101, arXiv:1711.05101.

[13] Lukosevicius, M. Echo state networks with trained feedbacks (2007).

[14] Maass, W., Natschläger, T., and Markram, H. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, **14** (2002), 2531. doi:10.1162/089976602760407955.

[15] Paszke, A., et al. Pytorch: An imperative style, high-performance deep learning library (2019). Available from: https://arxiv.org/abs/1912.01703, doi:10.48550/arXiv.1912.01703.

[16] Peng, B., et al. Rwkv: Reinventing rnns for the transformer era (2023). Available from: https://arxiv.org/abs/2305.13048, arXiv:2305.13048.

[17] Schrauwen, B., Wardermann, M., Verstraeten, D., Steil, J. J., and Stroobandt, D. Improving reservoirs using intrinsic plasticity. *Neurocomput.*, **71** (2008), 1159–1171. Available from: https://doi.org/10.1016/j.neucom.2007.12.020, doi:10.1016/j.neucom.2007.12.020.

[18] Sutton, R. S. The bitter lesson. http://www.incompleteideas.net/IncIdeas/BitterLesson.html (2019). Accessed: 2026-06-27.

[19] Triesch, J. A gradient rule for the plasticity of a neuron's intrinsic excitability. In *Artificial Neural Networks: Biological Inspirations – ICANN 2005* (edited by W. Duch, J. Kacprzyk, E. Oja, and S. Zadrożny), pp. 65–70. Springer Berlin Heidelberg, Berlin, Heidelberg (2005). ISBN 978-3-540-28754-4.

[20] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need (2023). Available from: https://arxiv.org/abs/1706.03762, arXiv:1706.03762.

[21] Yuan, J., et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (edited by W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar), pp. 23078–23097. Association for Computational Linguistics, Vienna, Austria (2025). ISBN 979-8-89176-251-0. Available from: https://aclanthology.org/2025.acl-long.1126/, doi:10.18653/v1/2025.acl-long.1126.

[22] Zhang, B. and Sennrich, R. Root mean square layer normalization. In *Advances in Neural Information Processing Systems* (2019).